



Las Americas Institute of Technology

Grupo no. 3

Sustentantes:

Deybby Rosario	2024-0504
Sarah Peña	2024-0506
Carlos Roa	2024-0510
Héctor Reyes	2024-0526

Materia:

Juegos Inteligentes

Docente:

Carlos Bienvenido Ogando Montas

Sección:

2025-C-3-1908-3242-TIA-603

Fecha:

29-10-2025

Tema:

Proyecto Reversi - Búsqueda adversarial

MiniMax en Reversi

1. Introducción

Reversi es un juego de mesa para dos jugadores. El objetivo del juego es capturar tantas piezas en forma de disco como sea posible del oponente, atrapándolas entre las tuyas. Los jugadores se turnan para colocar un disco (pieza del juego) en el tablero, con su color designado hacia arriba.

El objetivo de cada movimiento es atrapar uno o más discos de tu oponente entre dos de los tuyos. Cuando atrapas discos, puedes voltearlos y así invertir su color para que sean tuyos.

El juego termina cuando ambos jugadores han colocado todos sus discos en el tablero, o cuando no hay más movimientos legales posibles para ninguno de los jugadores. El jugador con más discos en el tablero es el ganador.

El siguiente proyecto de **Minimax Reversi** es una implementación del algoritmo **Minimax con Poda Alpha-Beta** en el juego de mesa **Reversi** o también conocido como (**Othello**). El sistema está diseñado para explorar y probar múltiples valores de heurísticas y diferentes tipos de jugadores. Ofrece tanto una interfaz interactiva para el juego humano-AI como un modo de prueba de algoritmos optimizado para la recopilación exhaustiva de métricas de rendimiento.

Característica	Detalle
Algoritmo Central	Minimax con Poda Alpha-Beta y Búsqueda de Profundización Iterativa .
Evaluación Heurística	Sistema de evaluación adaptativo por fase del juego (Apertura, Medio, Final).
Modos de Juego	Interactivo (Consola) y Pruebas de Algoritmos (Rápido, sin UI).
Métricas	Seguimiento de nodos expandidos, profundidad explorada, tiempo de ejecución y uso de RAM.

2. Arquitectura y Componentes Centrales

El sistema se organiza en capas arquitectónicas, con el motor de juego (ReversiBoard) actuando como el núcleo de la lógica y la IA (MinimaxPlayer) interactuando con este núcleo

para tomar decisiones.

Componentes Principales

El proyecto se sustenta en tres clases principales que definen la funcionalidad:

Componente	Archivo Fuente	Responsabilidad Principal	Métodos Clave
ReversiBoard	game/reversiBoard.py	Gestión del estado del tablero, aplicación de reglas del juego y evaluación heurística de la posición.	posible_movement, insert_play, evaluate, is_terminal
ReversiGame	game/reversiGame.py	Orquestación del juego, gestión de turnos, coordinación de jugadores y control de modos de ejecución.	play, play_for_algorithms, swap_turn, app
MinimaxPlayer	players/minimax.py	Implementación del algoritmo Minimax con Alpha-Beta para determinar el mejor movimiento.	play, _minimax,

```
===== Jugador 1: human, Jugador 2: minimax ===  
=== Reversi Game ===  
> Jugar <  
Seleccionar Jugadores  
Salir  
Presiona una tecla [W][S][ENTER])  
█
```

Flujo de Interacción

1. **ReversiGame** inicia el turno y solicita el movimiento al **current_player** (que puede ser un MinimaxPlayer, BadPlayer, GreedyPlayer, RandomPlayer o HumanPlayer).
2. **MinimaxPlayer** (la IA) invoca a su función `play(board)`.
3. **MinimaxPlayer** ejecuta el algoritmo `_minimax()`, que a su vez llama repetidamente a `ReversiBoard.childrens()` para generar los estados sucesores y a `ReversiBoard.evaluate()` para puntuarlos.
4. Una vez seleccionado, el movimiento (coordenada) se devuelve a **ReversiGame**.
5. **ReversiGame** llama a `ReversiBoard.insert_play()` para actualizar el estado del tablero.

```
AI juega en (5, 4)
Turno de Deybby (B) Profundidad: 2
Puntajes:
Rojas: 3 - Azules: 3
  0  1  2  3  4  5  6  7
-----
0 |  |  |  |  |  |  |  |
-----
1 |  |  |  |  |  |  |  |
-----
2 |  |  |  |  0  |  |  |
-----
3 |  |  |  0  0  |  |  |
-----
4 |  |  |  0  0  0  |  |
-----
5 |  |  █  █  █  █  █  |
-----
6 |  |  |  |  |  |  |  |
-----
7 |  |  |  |  |  |  |  |
-----

Movimientos posibles: [(5, 5), (6, 5), (4, 5), (2, 5), (3, 5)]
Que movimiento desea hacer?
Ingrese la coordenada (x,y) [salir] para salir: █
```

3. Características Clave de la IA

El sistema de IA se basa en la clase `MinimaxPlayer` e incorpora técnicas avanzadas para mejorar la profundidad de búsqueda y la calidad de la toma de decisiones.

3.1. Algoritmo Minimax Avanzado

- **Poda Alpha-Beta (`_minimax`):** Reduce significativamente la cantidad de nodos a explorar sin sacrificar la exactitud del resultado, optimizando el rendimiento.
- **Búsqueda de Profundización Iterativa (`play`):** El algoritmo inicia la búsqueda a

profundidad 2, y progresivamente aumenta la profundidad (3, 4, 5...) hasta que se agota el tiempo límite (`max_time`). Esto asegura que, incluso si el tiempo expira, siempre se tendrá el resultado de la búsqueda completa más profunda alcanzada.

3.2. Explicación de las Heurísticas Utilizadas

La clase `ReversiBoard` define 5 heurísticas principales que se utilizan para evaluar el tablero. Todas están diseñadas para devolver un valor normalizado entre **-100.0** y **+100.0**, donde un valor positivo favorece al jugador (`color`) y un valor negativo favorece al oponente.

`_corner_occupancy` (Ocupación de Esquinas)

- **Qué hace:** Evalúa quién controla las cuatro esquinas estratégicas del tablero: $(0, 0)$, $(0, 7)$, $(7, 0)$ y $(7, 7)$.
- **Cómo lo calcula:**
 1. Define una lista de las 4 coordenadas de las esquinas.
 2. Itera sobre esta lista, contando cuántas esquinas pertenecen al jugador (`my_c`) y cuántas al oponente (`op_c`).
 3. Aplica la fórmula: $100.0 * (my_c - op_c) / (my_c + op_c)$
- **Propósito:** Las fichas en las esquinas son las más valiosas del juego porque son **permanentes**: una vez que una ficha está en una esquina, nunca puede ser volteada por el oponente.

`_mobility` (Movilidad)

- **Qué hace:** Mide la ventaja en términos de la cantidad de movimientos válidos disponibles para cada jugador.
- **Cómo lo calcula:**
 1. Cuenta el número de movimientos posibles para el jugador (`my_moves`) llamando a `len(self.posible_movements(color))`.
 2. Cuenta el número de movimientos posibles para el oponente (`op_moves`).
 3. Aplica la fórmula: $100.0 * (my_moves - op_moves) / (my_moves + op_moves)$
- **Propósito:** Esta es una heurística estratégica clave. Tener una alta movilidad (muchas opciones de jugada) es bueno. Aún mejor es restringir la movilidad del oponente, dejándolo con pocas o ninguna jugada válida.

`_stable_discs` (Fichas Estables)

- **Qué hace:** Identifica y cuenta todas las fichas (no solo las esquinas) que ya no pueden ser volteadas por el oponente durante el resto de la partida.
- **Cómo lo calcula:** Este es el método más complejo.

1. `_stable_discs`: Recorre cada celda del tablero. Si hay una ficha, llama a `_is_stable(x, y)` para verificarla. Cuenta el total de fichas estables para el jugador (`my_s`) y el oponente (`op_s`).
 2. `_is_stable(x, y)`: Determina si una ficha en `(x, y)` es estable. Una ficha se considera estable si está "asegurada" en los 4 ejes (horizontal, vertical y las dos diagonales).
 3. `_line_stable_in_direction(...)`: Para que un eje esté "asegurado" (por ejemplo, el horizontal), la ficha debe formar parte de una línea ininterrumpida de su *propio color* hasta el borde del tablero en al menos una de las dos direcciones de ese eje (ej: toda la línea hasta el borde izquierdo O toda la línea hasta el borde derecho está llena de sus propias fichas).
 4. Finalmente, aplica la fórmula: $100.0 * (my_s - op_s) / (my_s + op_s)$
- **Propósito:** Esta es una versión mucho más precisa y potente de la heurística de esquinas. Mide el control "permanente" y real sobre el tablero.

`_coin_parity` (Paridad de Fichas)

- **Qué hace:** Mide la ventaja numérica más simple: la diferencia en la cantidad total de fichas que tiene cada jugador.
- **Cómo lo calcula:**
 1. Obtiene el conteo de fichas del jugador (`myc`) y del oponente (`opc`) usando el método `self.points()`.
 2. Aplica la fórmula: $100.0 * (myc - opc) / (myc + opc)$
- **Propósito:** Al final del juego, esta es la única heurística que importa (quien tiene más fichas gana). Sin embargo, al principio y a mitad del juego, es un indicador pobre, ya que tener muchas fichas no significa tener una buena posición.

`_positional_weights` (Pesos Posicionales)

- **Qué hace:** Asigna un valor estratégico fijo a cada una de las 64 casillas del tablero, basándose en la matriz `POSITION_WEIGHTS`.
- **Cómo lo calcula:**
 1. Recorre todo el tablero.
 2. Si una ficha pertenece al jugador, *suma* el valor de `POSITION_WEIGHTS[y][x]` a una puntuación total (`score`).
 3. Si la ficha pertenece al oponente, *resta* ese mismo valor.
 4. La matriz `POSITION_WEIGHTS` da valores muy altos a las esquinas (ej: 100), valores muy negativos a las casillas *adyacentes* a las esquinas (ej: -50, -20), y valores moderados a los bordes.
 5. Normaliza el `score` total dividiéndolo por la suma de todos los pesos absolutos en la matriz (`max_abs`) y multiplicando por 100.
- **Propósito:** Es una "regla general" que guía a la IA para que prefiera posiciones estratégicamente buenas (como las esquinas) y evite posiciones peligrosas (como las que "regalan" la esquina al oponente).

Cómo se Usan Juntas: `evaluate` y `_phase_weights`:

Estas heurísticas no se usan solas. La función principal `evaluate` las combina:

1. `evaluate(...)`: Es el cerebro que une todo. Calcula el valor de cada heurística activada (ej: `mob_value = _mobility(color)`).
2. `_phase_weights(...)`: Esta función crucial decide *cuánto importa* cada heurística dependiendo de la fase del juego (medida por `empty_cells`):
 - **Apertura (>= 40 vacías)**: Da más peso a `_mobility` (35%) y `_positional_weights` (30%). El conteo de fichas (`_coin_parity`) casi no importa (5%).
 - **Medio Juego (>= 15 vacías)**: El peso se balancea. `_mobility` (30%), `_stable_discs` (25%) y `_corner_occupancy` (25%) son las más importantes.
 - **Final (< 15 vacías)**: La estrategia cambia por completo. Ahora lo más importante es `_stable_discs` (40%) y `_coin_parity` (30%). La movilidad (`_mobility`) ya casi no importa (5%).

La puntuación final del tablero es una **suma ponderada**: $\text{Puntuación} = (\text{valor_mob} * \text{peso_mob}) + (\text{valor_corner} * \text{peso_corner}) + \dots$

3.3. Sistema de Evaluación Heurística Adaptativa

La función `ReversiBoard.evaluate()` utiliza una combinación ponderada de cinco heurísticas. Los pesos de estas heurísticas no son estáticos, sino que se ajustan dinámicamente según la **fase del juego** (determinada por el número de casillas vacías).

Heurística	Propósito	Ponderación en Fases Críticas
Ocupación de Esquinas	Control de las 4 esquinas del tablero (posiciones irreflejables).	Alta en Apertura y Medio debido a su gran valor estratégico.
Movilidad	Número de movimientos legales disponibles.	Alta en la Apertura para maximizar opciones.
Discos Estables	Conteo de piezas que no pueden ser volteadas.	Alta en el Final del juego.
Paridad de Monedas	Diferencia neta de piezas.	Alta en el Final del juego.
Pesos Posicionales	Valoración estática de las casillas del tablero.	Alta en la Apertura para establecer una buena estructura.

4. Modos de Ejecución y Métricas

El proyecto permite el análisis y la interacción a través de dos modos de ejecución distintos:

4.1. Modos de Ejecución

Modo	Acceso	Descripción
Interactivo	<code>ReversiGame.play(board)</code>	Incluye una Interfaz de Usuario (UI) basada en la consola con colores ANSI, un menú de selección de jugadores y un flujo de juego paso a paso, ideal para la interacción Humano-IA.
Pruebas de Algoritmos	<code>ReversiGame.play_for_algorithms(board)</code>	Optimizado para la velocidad. Sin UI ni pausas. Diseñado para competiciones rápidas de IA contra IA, con un enfoque en la recolección automática de métricas.

4.2. Métricas de Rendimiento

La clase `MinimaxPlayer` y el módulo `metrics.py` rastrean las siguientes variables clave para la evaluación del rendimiento de la IA:

- `nodes_expanded`: Cantidad promedio de nodos visitados por turno en una partida.
- `depth_explored`: Profundidad máxima promedio de búsqueda alcanzada dentro del tiempo límite.
- `execution_time`: Tiempo promedio de cómputo para la toma de decisión.
- `max_ram_usage`: Pico de consumo promedio de memoria RAM durante la búsqueda.

Estos datos se exportan a archivos `.csv` en el directorio `output/` para un análisis posterior.

output >  weights_6.csv

	opponent	winner	points	Total_points	turns	total_nodes_expanded	total_search_depth	total_running_time	total_max_ram_usage
1	weights_4	weights_6	52	59	30	41.90	1.40	1.02	0.56
2	weights_5	weights_5	18	63	29	39.93	1.41	1.02	0.57
3	greedy	weights_6	44	64	30	43.57	1.23	1.02	0.57
4	random	weights_6	50	64	30	33.40	1.20	1.02	0.51
5	minimax_base		32	64	30	36.83	1.13	1.03	0.59
6	weights_1	weights_1	15	64	30	33.60	1.10	1.03	0.58
7	weights_2	weights_2	30	63	30	40.93	1.27	1.02	0.49
8	bad_player	weights_6	43	64	30	35.60	1.03	1.02	0.50
9	weights_3	weights_6	37	64	30	35.20	1.07	1.02	0.54
10	heuristic_1	weights_6	38	63	30	32.83	1.03	1.02	0.50
11	heuristic_3	heuristic_3	23	63	29	32.52	1.86	1.02	0.57
12	heuristic_2	weights_6	41	63	30	36.37	1.77	1.02	0.54
13	heuristic_4	weights_6	35	61	30	44.33	1.27	1.01	0.60
14	heuristic_5	weights_6	38	64	30	42.43	1.03	1.02	0.67
15	time_3	time_3	0	52	19	51.47	3.47	1.01	0.66
16	time_10	time_10	14	64	30	85.30	1.97	1.01	0.91
17	Deybby	weights_6	34	62	30	154.60	2.33	1.00	1.17
18	weights_7	weights_7	24	64	30	48.33	1.47	1.01	0.59

5. Organización de Archivos del Proyecto

El proyecto está estructurado modularmente para separar la lógica de negocio (juego) de las implementaciones de la inteligencia artificial (jugadores).

MiniMax-in-reversi-game/

```

├── main.py          # Punto de entrada de la aplicación, lanza ReversiGame.app()
├── game/            # Paquete de Lógica Central del Juego
│   ├── reversiBoard.py    # Tablero, reglas, y lógica de evaluación heurística
│   ├── reversiGame.py    # Orquestación y gestión del flujo del juego
│   ├── tokens.py         # Definición de fichas (Token)
│   ├── menu.py          # Sistema de Menú para la consola
│   ├── control.py       # Gestión de la entrada de teclado
│   ├── metrics.py       # Recolección y reporte de métricas
│   └── color.py         # Códigos de color ANSI para la UI
├── players/         # Implementaciones de la IA y Jugadores
│   ├── player.py        # Clase base abstracta (ABC) para todos los jugadores
│   ├── minimax.py       # Implementación avanzada de Minimax con Alpha-Beta
│   ├── humanPlayer.py   # Interfaz para jugador humano
│   ├── greedy.py        # IA de estrategia codiciosa (profundidad 1)
│   ├── randomPlayer.py  # IA que selecciona movimientos aleatorios
│   └── badPlayer.py     # IA débil para fines comparativos
├── output/          # Directorio de salida para reportes de rendimiento (.csv)
├── genetic_algorithm # Algoritmo Genético para optimizar configuracion de heurísticas
├── gens_analysis    # Analizar genes
└── test_minimax.py  # Evaluar algoritmos y heurísticas de forma masiva

```

Para Empezar

Para iniciar la aplicación, simplemente ejecute `main.py` desde la raíz del proyecto. El menú de la aplicación le permitirá seleccionar los jugadores (Humano, Minimax, Greedy, etc.) y comenzar una partida.

Evaluación de Configuraciones Heurísticas en un Agente Minimax para Reversi (IA vs Humano)

El objetivo del experimento fue analizar el rendimiento de diferentes configuraciones de pesos heurísticos aplicadas al algoritmo de búsqueda adversarial **Minimax** en el juego *Reversi*. Cada configuración representó una distinta ponderación de las heurísticas: **corner**, **mobility (mob)**, **stability (stable)**, **parity**, y **positional value (pos)**, ajustadas de forma independiente para las tres fases del juego: *apertura*, *medio juego* y *final*.

Para evitar sesgos subjetivos durante la evaluación, se implementó una metodología de prueba **ciega**. Se construyó una lista con todas las configuraciones de heurísticas (tanto las versiones puras como las ponderadas) y se aplicó un `random.shuffle()` antes de cada partida. De esta manera, el evaluador humano (el jugador) no conocía contra cuál configuración competía en cada enfrentamiento, garantizando que su estilo de juego no se adaptara inconscientemente a un comportamiento específico de la IA.

Cada partida fue ejecutada bajo las mismas condiciones:

- Tiempo máximo por jugada: **1 segundo**.
- Misma versión del motor *Minimax*.
- Tablero inicial estándar (8x8).
- Sin ventaja inicial de color (negro/blanco asignado aleatoriamente).

Al finalizar cada partida, se registraron las observaciones cualitativas sobre el estilo de juego de la IA, las fases donde mostró mayor fortaleza o debilidad, y el resultado final en puntuación.

2. Configuraciones analizadas

Se evaluaron once configuraciones distintas del algoritmo *Minimax*:

- **Heurísticas puras:** `heuristic_1` a `heuristic_5`, con pesos crecientes sobre las métricas (desde solo *corner* hasta la combinación completa de todas las heurísticas).
- **Pesos personalizados:** `weights_1` a `weights_6`, generadas con valores continuos entre 0 y 1 para cada heurística, variando según la fase del juego.

Estas configuraciones buscaban contrastar el rendimiento entre estrategias simplificadas (con pocas variables de decisión) y combinaciones complejas donde las heurísticas

interactúan en distintas proporciones.

3. Resultados y observaciones

Heurísticas puras

- **Heuristic_1 (solo corner):**
Priorizó fuertemente las diagonales, lo que dejó disponibles los bordes al jugador humano. Aunque inició con ventaja, su estrategia fue fácilmente contrarrestable. Resultado: derrota amplia.
- **Heuristic_2 (corner + mobility):**
Mostró una estrategia más adaptable. Aprovechó los turnos para limitar los movimientos del jugador humano y remontó al final. Ganó por **5 puntos**.
Indicó un comportamiento propio de un modelo que entiende el valor de la restricción del oponente.
- **Heuristic_3 (corner + mob + stable):**
Valoró excesivamente los bordes, cayendo en rigidez estratégica. Aunque mantuvo la paridad en puntos en el medio juego, logró dejar al jugador sin movimientos en momentos clave, tomando tres bordes y dominando la posición. Ventaja significativa para la IA.
- **Heuristic_4 (corner + mob + stable + parity):**
Descuidó esquinas, lo que permitió al jugador humano posicionarse mejor. Aunque el juego fue parejo, perdió en el cierre. Muestra un desequilibrio entre priorización posicional y control de estabilidad.
- **Heuristic_5 (todas las heurísticas = 1):**
Exhibió un comportamiento más balanceado. En el inicio cedió algunos bordes pero logró mantener buena puntuación total. Aprovechó un error del jugador para tomar una esquina. Resultado parejo, demostrando que el exceso de ponderación uniforme no siempre produce el mejor balance estratégico.

Pesos personalizados

- **Weights_1:**
Estrategia equilibrada pero conservadora. No priorizó bordes ni esquinas agresivamente. Mantuvo paridad gran parte de la partida, pero cedió ventaja en el tramo final.
- **Weights_2:**
Mostró solidez. Valoró las esquinas y penalizó los movimientos del jugador, reduciendo drásticamente sus opciones. Resultado favorable a la IA.

- **Weights_3:**
Desempeño débil frente al jugador humano. Aunque logró algunas jugadas efectivas, no consolidó control posicional. Derrota abultada (**50 a 14**).
- **Weights_4:**
No priorizó esquinas ni bordes, pareciendo centrarse en la maximización del conteo de fichas (*score-based play*). Dejó sin movimientos al jugador en el tramo final y ganó. Refleja un estilo “numérico” más que estratégico.
- **Weights_5:**
Juego muy equilibrado. La IA capitalizó un error del jugador para revertir la partida y ganar por escaso margen. Denota consistencia pero poca agresividad inicial.
- **Weights_6:**
Cedió bordes pero impidió el acceso a las esquinas. Logró bloquear movimientos en la fase final y ganó por solo **2 puntos**, reflejando un estilo de control posicional moderado.

4. Análisis comparativo

El desempeño general sugiere que:

- Las configuraciones que combinan **corner**, **mobility** y **parity** de manera balanceada (como **heuristic_2**, **weights_2** y **weights_6**) logran mejores resultados al **restringir los movimientos del oponente** sin comprometer el control de esquinas.
- Las estrategias con exceso de peso en una sola métrica (como **heuristic_1** o **heuristic_3**) presentan **comportamientos predecibles y explotables**.
- Los algoritmos que priorizan la **movilidad** tienden a ganar partidas largas al dejar al oponente sin opciones, incluso si inician en desventaja.
- El uso de **pesos adaptativos por fase de juego** (caso de los **weights_#**) introduce una dinámica más humana, ya que el agente cambia de enfoque entre la apertura y el final.

5. Conclusiones

El test permitió una comparación objetiva del rendimiento heurístico, demostrando que:

- No existe una configuración universalmente dominante.
- La **movilidad** y la **paridad** son factores decisivos en la victoria.
- No hay que ser el mejor jugador para crear al mejor jugador.